



# Reporte Técnico de Actividades Práctico-Experimentales Nro. 001

## 1. Datos de Identificación del Estudiante y la Práctica

|                                              |                                                                                                                                                            |
|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Nombre del estudiante(s)</b>              | Boris Israel Rengel Japon                                                                                                                                  |
| <b>Asignatura</b>                            | Desarrollo Basado en Plataformas                                                                                                                           |
| <b>Ciclo</b>                                 | 5 A                                                                                                                                                        |
| <b>Unidad</b>                                | 2                                                                                                                                                          |
| <b>Resultado de aprendizaje de la unidad</b> |                                                                                                                                                            |
| <b>Práctica Nro.</b>                         | 001                                                                                                                                                        |
| <b>Título de la Práctica</b>                 | Implementar un servicio REST con Node.js                                                                                                                   |
| <b>Nombre del Docente</b>                    | Edison Leonardo Coronel Romero                                                                                                                             |
| <b>Fecha</b>                                 | Viernes 3 de octubre                                                                                                                                       |
| <b>Horario</b>                               | 07h30 – 10h30                                                                                                                                              |
| <b>Lugar</b>                                 | Laboratorio Computación aplicada<br>Laboratorio Desarrollo de Software<br>Laboratorio de redes y Sistemas Operativos<br>Laboratorio Virtual<br>EVA<br>Aula |
| <b>Tiempo planificado en el Sílabo</b>       | 3 horas                                                                                                                                                    |

## 2. Objetivo(s) de la Práctica

Comprender y aplicar los conceptos de HTTP/HTTPS, métodos REST, códigos de estado y CORS desde el entorno del frontend.

Implementar una página web responsiva y semántica que realice peticiones simuladas a un servidor (real o mock).

Registrar, analizar y documentar los resultados obtenidos mediante herramientas de desarrollo. Fortalecer la capacidad de contrastar la teoría con la ejecución real dentro del proyecto integrador.

## 3. Materiales, Reactivos, Equipos y Herramientas

CVS Code o editor de código equivalente.

Archivo HTML/CSS/JS base del estudiante.

Servicio de prueba: cargada previamente en NotebookLM

Documentación (HTTP/REST/CORS/W3C).

## 4. Procedimiento / Metodología Ejecutada

aplicación rápida de métodos HTTP: GET, POST, PUT, DELETE.

Explicación del objetivo de la práctica.

## 5. Resultados

The image displays a web browser window with a REST client application. The main heading is "Prueba de Peticiones HTTP" (HTTP Request Test), with a subtitle "Simulación de métodos REST con JSONPlaceholder" (Simulation of REST methods with JSONPlaceholder). The interface is divided into several sections:

- Realizar Peticiones (Perform Requests):** A horizontal bar containing four buttons: "GET - Obtener" (green), "POST - Crear" (blue), "PUT - Actualizar" (orange), and "DELETE - Eliminar" (red).
- Resultados (Results):** A section showing the outcome of a request. It indicates a "Petición POST exitosa" (Successful POST request) with a status of 201. A dark blue box displays the JSON response: 

```
{  "title": "Nuevo Post desde mi aplicación",  "body": "Este es un post de prueba creado con el método POST",  "userId": 1,  "id": 101}
```
- Información de la Petición (Request Information):** A table-like structure showing details of the performed request:

|                   |                                            |
|-------------------|--------------------------------------------|
| Método:           | POST                                       |
| URL:              | https://jsonplaceholder.typicode.com/posts |
| Código de Estado: | 201                                        |

At the top right, a grey progress bar shows "33%". Below the main interface, a second, partially visible section shows a "Petición GET exitosa" (Successful GET request) with a status of 200, displaying a JSON response for a post with id 1.



## Prueba de Peticiones HTTP

Simulación de métodos REST con JSONPlaceholder

### Realizar Peticiones

GET - Obtener

POST - Crear

PUT - Actualizar

DELETE - Eliminar

### Resultados

✓ Petición DELETE exitosa

Status: 200

```
{}
```

### Información de la Petición

Método: DELETE

URL: https://jsonplaceholder.typicode.com/posts/1

Código de Estado: 200

Tiempo de Respuesta: 138 ms

CORS: No especificado

Timestamp: 21/11/2025, 9:48:41

Observaciones: Petición exitosa

### Registro de Peticiones

| Método | URL                                          | Código | Tiempo (ms) | CORS            | Observaciones    |
|--------|----------------------------------------------|--------|-------------|-----------------|------------------|
| DELETE | https://jsonplaceholder.typicode.com/posts/1 | 200    | 138 ms      | No especificado | Petición exitosa |
| PUT    | https://jsonplaceholder.typicode.com/posts/1 | 200    | 331 ms      | No especificado | Petición exitosa |
| POST   | https://jsonplaceholder.typicode.com/posts   | 201    | 262 ms      | No especificado | Recurso creado   |
| GET    | https://jsonplaceholder.typicode.com/posts/1 | 200    | 1047 ms     | No especificado | Petición exitosa |

Exportar a Markdown

## 6. Preguntas de Control

**¿Qué diferencia existe entre un código de estado 200, 201, 400 y 500?**

- **200 (OK):** Petición exitosa, el servidor devolvió la información solicitada
- **201 (Created):** Recurso creado exitosamente (común en POST)
- **400 (Bad Request):** Error del cliente, petición mal formada
- **500 (Internal Server Error):** Error del servidor al procesar la petición

**¿Qué función cumple CORS en una aplicación web?**

CORS (Cross-Origin Resource Sharing) es un mecanismo de seguridad que permite o restringe peticiones HTTP desde un dominio diferente al del servidor. Protege contra accesos no autorizados desde otros orígenes.

**¿Cuál es la diferencia entre request headers y response headers?**

- **Request Headers:** Información enviada por el cliente al servidor (credenciales, tipo de contenido aceptado, etc.)
- **Response Headers:** Información enviada por el servidor al cliente (tipo de contenido, políticas CORS, cache, etc.)

**¿Por qué es importante documentar los tiempos de respuesta?**

Porque permite:

- Identificar cuellos de botella
- Optimizar el rendimiento
- Establecer SLAs (Service Level Agreements)
- Detectar problemas de red o servidor
- Mejorar la experiencia de usuario

**¿Qué riesgos tiene exponer peticiones sin validar en el frontend?**

- Inyección de código malicioso
- Exposición de datos sensibles
- Ataques XSS (Cross-Site Scripting)
- Manipulación de datos
- Falta de control sobre la información enviada

**¿Qué consideraciones de seguridad se deben mantener al exponer endpoints entre servicios?**

- Implementar autenticación y autorización
- Validar datos en el backend
- Usar HTTPS
- Configurar CORS correctamente
- Rate limiting para prevenir abusos
- Sanitizar inputs
- No exponer información sensible en respuestas
- Implementar logs y monitoreo

## 7. Conclusiones

- Durante el desarrollo de la práctica fue posible comprender de manera efectiva cómo funcionan los protocolos HTTP y HTTPS, así como la



---

estructura y propósito de los métodos REST. Al interactuar con peticiones simuladas desde el frontend, se evidenció la importancia de manejar correctamente los verbos de solicitud, los encabezados y los códigos de estado para asegurar una comunicación clara y confiable entre cliente y servidor.

- La identificación de los códigos de estado permitió interpretar adecuadamente las respuestas del servidor y detectar posibles fallos o restricciones. Además, experimentar con CORS ayudó a entender cómo los navegadores implementan políticas de seguridad para evitar solicitudes no autorizadas y cómo configurarlas correctamente para permitir el funcionamiento de la aplicación web.

## **8. Recomendaciones**

## **9. Bibliografía / Referencias**

## **10. Anexos**